SPECULATIVE DECODING ACROSS CONSUMER SYSTEMS: GAINS, FAILURES, AND RUNTIME COSTS

Anonymous Authors[1]

ABSTRACT

Favorable local indicators make speculative decoding promising, but how reliably do they translate into integrated gains on consumer systems? We characterize four frozen hardware/runtime configurations under greedy token equivalence, comparing decode execution after initial prompt prefill. On the same MacBook Air M4, a representative workload is exact but reaches only 0.842× speedup (95% CI [0.809, 0.877]), whereas a separately preregistered HumanEval workload is exact and reaches 1.387× [1.351, 1.422] with 90/90 exact pairs. We diagnose two concrete failures of local promise: correctness qualification excludes cheap verification paths on RTX 5070 Ti Laptop and Arc B570, while the realized 358H state interface erases a favorable isolated NPU draft advantage. The exact 358H CPU and NPU paths reach only 0.734× and 0.352×, with complete target-side work and replay plus proposal dominating their respective costs. Favorable local indicators therefore do not reliably predict gains across these tested configurations; the two diagnoses explain important failures without constituting an exhaustive theory or a measured runtime intervention.

Manuscript v0.2—hardware-complete / runtime-reserved. Title and abstract provisional.

1 INTRODUCTION

Speculative decoding promises lower serial decoding latency by using a cheaper draft model to propose tokens that a target model validates together (Leviathan et al., 2023). High draft acceptance, fast small-model decoding, and cheap batched target execution can each make an implementation look promising. For a consumer system, however, the relevant outcome is the gain obtained when those parts run together. How reliably does that local promise translate across workloads and runtime stacks?

We investigate this question on MacBook Air M4, Core Ultra 9 275HX with RTX 5070 Ti Laptop, Core i5-13400 with Arc B570, and Core Ultra X7 358H with Arc B390 and CPU/NPU drafting. The observations differ sharply even within a platform. The representative M4 workload is token-exact but reaches only 0.842× speedup; a separately preregistered HumanEval workload on the same M4 is exact and reaches 1.387×. The RTX batched target has a favorable forward microbenchmark but fails the frozen correctness gate. On 358H, favorable isolated NPU draft throughput accompanies only 0.352× integrated speedup. These outcomes do not support treating profitability as a simple hardware capability.

Our primary question is: *under what conditions do favorable local indicators translate into integrated speculative-decoding gains on the tested consumer configurations?* We compare matched decode execution under strict greedy token equivalence, excluding loading and initial prompt prefill. Hardware, workload, and runtime are not independently controlled. The M4 contrast is workload-conditioned; differing session conditions prevent attributing it to workload alone.

Favorable local indicators do not translate reliably into gains across these configurations. We diagnose two concrete failure modes. On RTX 5070 Ti Laptop, correctness qualification removes cheap batching; the tested exact serialized controls cost more. B570 serialization improves agreement without qualifying the full workload. On 358H, the realized state interface erases the isolated NPU draft advantage, while complete target-side work dominates CPU drafting. These diagnoses explain important failures without exhausting possible causes or explaining the M4 contrast.

Our contributions are a cross-platform and cross-workload characterization retaining positive, negative, and correctness-blocked outcomes; controlled evidence that qualification changes the usable verification path; direct 358H evidence connecting an isolated draft advantage to unfavorable stateful integration; and an auditable accounting discipline separating microbenchmarks, commit-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

ted progress, correctness status, and realized costs. Prior work already studies performance, heterogeneous placement, cost/commit accounting, and output reproducibility (Zhang et al., 2025; Lv et al., 2025; Liu et al., 2026; Kim et al., 2026; Gond et al., 2026). We contribute the bounded consumer-system observations and diagnoses, not a new universal theory. The hardware argument is complete independently of any subsequent runtime intervention.

2 Experimental Question and Measurement Discipline

We investigate the gap between favorable component measurements and integrated decode outcomes. Acceptance describes candidate agreement, isolated draft throughput describes one execution boundary, and target-forward batching describes another. None alone measures the complete speculative loop. We use an output contract and an accounting lens to make those boundaries explicit before comparing the frozen campaigns.

2.1 A path-qualified output contract

A greedy speculative round proposes a draft sequence, verifies it with the target, and commits the matching prefix followed by a correction or bonus token when the remaining output budget permits. Rejected candidate state must be removed or reconstructed consistently. Our contract requires generated token IDs to equal the matched target-only greedy output under the frozen configuration. It is an empirical property of a model, runtime, execution path, and workload, not a property inferred from the algorithm name.

We call a tested path *admissible* only within the workload scope for which that contract passes. A failed pair blocks a workload-global optimization claim even if the timing is favorable. A small qualification probe establishes eligibility for broader testing, not universal equivalence. We retain blocked-path timings as diagnostics with their correctness status visible. This deliberate contract is stricter than an approximate-quality deployment objective; HumanEval token equality does not establish functional correctness or pass@k.

2.2 Evidence classes

Primary results answer a frozen workload question after its correctness gate. **Secondary** results are registered variants or explicitly bounded follow-up controls. **Mechanism-supported** claims use path controls or timed regions. **Diagnostic** values describe correctness-blocked or otherwise restricted paths. These labels describe permitted inference, not just measurement precision. In particular, the exact RTX serialized controls are secondary evidence; they do not replace its blocked primary fast path. B570 follow-ups remain diagnostic after the formal stop.

2.3 Complete round costs

Let T_b be the matched target-only cost per output token. The realized draft time T_d^{real} denotes the realized draft-subsystem time per round, including any prefix replay performed by its interface. The complete target time $T_{\text{target,round}}^{\text{real}}$ includes all measured target-side round work: candidate evaluation and any correction, bonus, or rollback work inside that region. Residual synchronization, transfer, orchestration, and bookkeeping time not included in either region is T_x. Let C count *all* committed output tokens. With consistent timing and aggregation boundaries,

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}, \tag{2}$$

where each denominator term is normalized by T_b. Break-even requires $C > D_{\text{real}} + R_{\text{target}} + X$. These identities organize measurements; they are not a new profitability theory. Cost/commit decompositions already appear in runtime-controlled studies (Kim et al., 2026).

At an aggregate level, round means and matched total-time/token accounting must use the same weights. A ratio of total times need not equal a geometric mean of prompt speedups. Likewise, accepted draft tokens per round omit correction or bonus output and cannot automatically be used as C. We do not infer unmeasured replay-only, transfer-only, or correction-only components by subtraction from incompatible experiments.

2.4 A separate target-forward proxy

For context n and path e, define

$$V_k^{\text{micro}}(n, e) = \frac{T_{\text{target forward}}(k, n, e)}{T_{\text{target decode}}(1, n)}. \tag{3}$$

This microbenchmark asks whether a target can process a causal block near the cost of a single decode. It omits draft generation and can omit target correction/bonus and state-management work. Consequently V_k^{micro} is not automatically R_{target}. Likewise, isolated draft throughput cannot substitute for realized stateful draft cost. These distinctions guide diagnosis after the observed outcomes; they do not supply a universal explanation for those outcomes.

Table 1. Frozen platform and runtime scope. All final target models are Qwen3-8B Q8_0. All campaigns use greedy generated-token equivalence to a matched target-only reference. Hardware and software co-vary.

System	Runtime / target backend	Draft model / placement	Evidence scope
MacBook Air M4	llama.cpp b10809 / Metal	Qwen3-0.6B Q8_0 / Metal	Representative 24Q and separate HumanEval30
Core Ultra 9 275HX + RTX 5070 Ti Laptop	llama.cpp b10839 / CUDA	Qwen3-0.6B Q8_0 / CPU or same GPU	Blocked fast path; exact serialized controls
Core i5-13400 + Arc B570	llama.cpp b10839 / Vulkan0	Qwen3-0.6B Q8_0 / B570	Formal gate failure; later diagnostics
Core Ultra X7 358H + Arc B390	llama.cpp b10839 / Vulkan0; OpenVINO draft	Qwen3-0.6B symmetric INT4 / CPU or NPU	Closed Phase 3; direct stateful decomposition

3 Experimental Methodology and Platforms

3.1 Frozen systems and workloads

Table 1 summarizes the completed campaigns. Final comparisons process one request at a time (sequence batch one). The final target is Qwen3-8B Q8_0; the native draft is Qwen3-0.6B Q8_0. The 358H heterogeneous adapter instead uses the qualified symmetric INT4 OpenVINO draft. We retain this quantization difference rather than describing the stacks as identical. Earlier M4 MLX-LM precision/correctness failures belong to qualification history and are distinct from the final llama.cpp Metal path.

The M4 representative sample freezes 24 previously unseen Spec-Bench prompts (Xia et al., 2024), with four from each of six workload groups and prior diagnostic IDs excluded. It uses three randomized paired repetitions. A separate preregistration freezes 30 HumanEval IDs (Chen et al., 2021) before prompt download or inspection; fixed $k = 2$ is the primary condition. The registered confidence-threshold conditions remain secondary. The subsequent HumanEval comparisons retain the frozen code-workload scope, three paired repetitions, greedy decoding, and a 128-token cap. They do not execute generated code.

Primary timing compares matched decode regions, excluding model loading and initial prompt prefill. In the native campaigns we use the frozen runtime decode counters; 358H explicitly times the complete post-prefill speculative loop. Replaying the committed prefix *inside* that loop remains charged to drafting. Thus the measurements are conditional decode costs, not whole-request latency including startup. Target-forward controls have their own narrower timing boundary. Different runtime versions, quantization, and device paths prevent a hardware-only causal comparison.

3.2 Statistical estimators and uncertainty

For each prompt we reduce its three paired baseline/speculative ratios, then report the geometric mean across prompts. M4 and RTX use a within-prompt geometric mean, as does the B570 diagnostic analysis; 358H uses a within-prompt median. Intervals are 95% prompt bootstraps. M4 HumanEval uses 100,000 resamples; the other reported campaigns use 10,000. Frozen seeds and endpoint conventions are given in the separate appendix. We preserve campaign-specific estimators and finite-bootstrap input order rather than harmonizing them after observing outcomes.

These intervals quantify prompt sampling conditional on the campaign. They do not encompass all session, temperature, or runtime-state uncertainty. Randomized adjacent pairing reduces drift but does not remove it. We therefore keep the representative and code-conditioned M4 studies separate, including their differing session conditions.

3.3 Provenance and energy boundaries

Frozen definitions, processed summaries, scripts, and archived evidence anchor every quantitative claim in an internal ledger. Windows raw per-run provenance is weaker than the other completed platforms, although committed pair tables and mechanism captures support recomputation. The 358H reconstructed draft source is `STRONGLY_SUPPORTED`; exact historical upstream identity is not proven. The header-only blocked-preflight derivative is excluded from final analysis.

Energy is interpreted only within its qualified sensor boundary. M4 measures package aggregate excluding separate DRAM; B570 measures adapter board energy. RTX board and CPU-package domains are not a common whole-system boundary, and its serialized HumanEval follow-up lacks an admissible formal energy comparison. No end-to-end Phase-3 energy claim is made for 358H. We do not rank platform energy efficiency.

Table 2. Headline decode outcomes. Speedup is baseline/speculative time, aggregated by the campaign's frozen prompt estimator. Brackets are 95% prompt-bootstrap CIs. Exact pairs and faster prompts are separate metrics. B570 timing is diagnostic, despite a reported CI.

Path / workload	Speedup [95% CI]	Exact pairs	Faster prompts	Class
M4 representative 24Q	0.842 [0.809, 0.877]	72/72	1/24	Primary
M4 HumanEval fixed-$k = 2$	1.387 [1.351, 1.422]	90/90	30/30	Primary
CPU to RTX 5070 Ti Laptop	0.404 [0.390, 0.418]	90/90	0/30	Second.
RTX 5070 Ti Laptop to itself	0.650 [0.629, 0.673]	90/90	0/30	Second.
B570 serialized fallback	0.3087 [0.2927, 0.3246]	60/90	0/30	Diag.
358H CPU draft	0.734 [0.693, 0.771]	90/90	0/30	Primary
358H NPU draft	0.352 [0.333, 0.369]	90/90	0/30	Primary

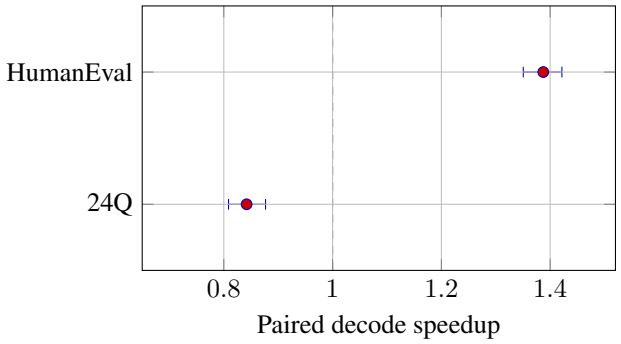


Figure 1. Does profitability carry across workloads on the same M4? Frozen 24Q is negative (72/72 exact); separately preregistered HumanEval is positive (90/90 exact). Bars are campaign-specific 95% prompt-bootstrap intervals. Both use the qualified Metal path. Workload and session conditions differ, so the contrast does not isolate workload causality.

4 Gains Do Not Translate Uniformly

The frozen outcomes establish the empirical puzzle before we examine its mechanisms. Table 2 keeps correctness and evidence class beside speed: a blocked path is a diagnostic observation, not a slower admissible optimization.

4.1 M4: workload-conditioned observed regimes

The representative frozen M4 sample produces 0.842× speedup (95% CI [0.809, 0.877]), with 72/72 exact pairs and only 1/24 prompts faster. The separately preregistered HumanEval fixed-$k = 2$ study produces 1.387× [1.351, 1.422], with 90/90 exact pairs and 30/30 prompts faster. Figure 1 retains both; the positive code-conditioned result does not replace or pool with the negative representative result.

The same M4 supports both an exact profitable observed regime and an exact unprofitable one. Profitability is therefore not a simple hardware capability property in these studies. It does not establish general M4 profitability or isolate unified memory as its cause. These are workload-conditioned observed regimes, not a controlled estimate of workload causality: workloads and session conditions differ. The earlier discovery effect size is not promoted as a confirmatory estimate. Historical MLX failures also do not negate the final qualified Q8_0 Metal result.

For primary fixed-$k = 2$ HumanEval, the package-efficiency ratio (baseline/speculative J per output token) is 1.013× [0.990, 1.034]. The interval includes one: energy is neutral within uncertainty. The registered $p_{\min} = 0.9$ condition has secondary energy evidence, discussed separately in the appendix. The preregistered corrected synthetic ceiling was not executed. We retain that deviation rather than treating the earlier synthetic diagnostic as its replacement.

4.2 Across stacks: negative and blocked outcomes

On RTX 5070 Ti Laptop, the serialized CPU-to-RTX control reaches 0.404× [0.390, 0.418], while the same-GPU draft control reaches 0.650× [0.629, 0.673]. Both are 90/90 exact and accelerate zero of 30 prompts. The higher speed of same-GPU placement shows that placement and associated integration costs matter, but its continued loss rules out transfer as the sole explanation.

On Arc B570, neither the fast path nor the serialized fallback supports a workload-global admissible optimization result. The latter's diagnostic speedup is 0.3087× [0.2927, 0.3246], with zero of 30 prompts faster. That interval describes timing on a partially mismatching path; its precision does not turn it into an admissible speed claim. The formal failure and stop rule remain the platform result. Board-energy observations likewise remain diagnostic and boundary-limited.

On Core Ultra X7 358H, the closed Phase-3 heterogeneous runner is exact on both CPU and NPU drafting: 90/90 pairs per path, with zero of 30 prompts faster. CPU achieves 0.734× [0.693, 0.771]; NPU achieves 0.352× [0.333, 0.369]. These exact negative outcomes coexist with favorable isolated draft measurements. Section 5 investigates which measured boundaries account for the failed

Table 3. Verification/admissibility mechanism. Proxy ranges span measured contexts, not confidence intervals. Full target-round R_{target} is not exposed by these microbenchmarks.

Path	V_2^{micro}	Qualification
M4 Metal	1.02–1.08	Frozen workloads exact
RTX batched	0.98–1.01	Correctness-blocked
RTX serial	2.14–2.43	Exact secondary control
B570 fast	1.24–1.64	45/90 exact; diagnostic
B570 serial	1.98–1.99	60/90 exact; diagnostic

local promise.

5 Why Favorable Local Indicators Fail

The outcome overview motivates two diagnoses supported by path controls and direct timing. They apply to the identified configurations, not to every platform or every negative result. We do not force the M4 contrast into either explanation.

5.1 Correctness changes the usable execution path

Cheap physical batching is not sufficient. The final M4 Metal path combines exact frozen-workload outputs with a target-forward proxy near one baseline step. Across its tested contexts, V_2^{micro} is approximately 1.02–1.08. This is a batching opportunity, not a complete round decomposition, and does not by itself explain whether a workload is profitable.

On RTX 5070 Ti Laptop, the batched CUDA target also has near-one-step two-token forward cost: $V_2^{\text{micro}} = 1.009$ and 0.983 at the two measured depths. The intended fast HumanEval execution nevertheless fails the frozen token-equivalence gate. Its timings cannot support an admissible optimization claim. The serialized control changes the micro-batch to one and raises the measured proxy to 2.431 and 2.137; the full frozen exact-control workload passes. Figure 2 therefore relates a physical opportunity to its empirically qualified status, rather than ranking GPUs.

These controls are deliberately secondary. They qualify a slower path after the native fast attempt failed; they are not a rescued primary success. The verification microbenchmark explains lost batching opportunity, while the complete end-to-end controls include drafting and integration. We cannot recover a full R_{target} decomposition by substituting the forward-only ratio into Equation 2.

A bounded numerical divergence. The frozen CUDA reproducer repeats the HumanEval/106 divergence in three of three runs at the same captured location. Sequential and serialized execution select token 2790, whereas batched execution selects 2629. A separate same-commit diagnostic build captures the corresponding score-margin change from $+0.080948$ to -0.126457. That numerical instrumentation does not replace the original formal binary or its measurements.

The evidence supports stable batch-shape-dependent greedy argmax divergence in this llama.cpp/CUDA/Q8_0 configuration. It does not identify a universal CUDA failure or a specific responsible kernel. Floating-point and batch-dependent output differences are established research topics (Yuan et al., 2025; Gond et al., 2026); the systems consequence here is that the cheap path is removed from the set admitted by the study's contract.

Serialization does not establish exactness. B570 provides a complementary boundary. Its formal v2 correctness gate fails, so the preregistered stop rule prevents a formal verification/performance claim from that execution. Later fast-path diagnostics yield 45/90 exact pairs. The unique serialized fallback improves this to 60/90, with 20/30 wholly exact prompts, but still fails workload-global equality. Its V_2^{micro} is approximately 1.983–1.992 and remains diagnostic.

A smaller serialized probe had passed before the full workload exposed mismatches. Passing such a gate is evidence for its tested scope, not proof of broader correctness. We therefore use "serialized V_2^{micro}" or "V2_serialized" for this B570 path. An execution property must not be renamed as an exactness guarantee.

This negative control matters because it prevents an attractive simplification: selecting serialized verification is not by itself a sufficient correctness policy. Qualification must cover the actual path and intended workload. Exact subsets may be descriptively useful, but conditioning on those subsets after the run cannot establish global performance.

5.2 Isolated drafting versus stateful integration

On Core Ultra X7 358H, the isolated NPU draft has higher median steady-state throughput than CPU at the tested contexts, with a larger advantage at longer contexts. This local indicator does not survive integration: both qualified paths are exact but unprofitable, and the NPU path is slower. The measured regions reveal different costs on the two paths. The direct round records show mean committed progress $C = 2.4631$ and 2.4584, respectively, approximately 2.46. This is complete committed output, not the accepted-draft counter used in older native records.

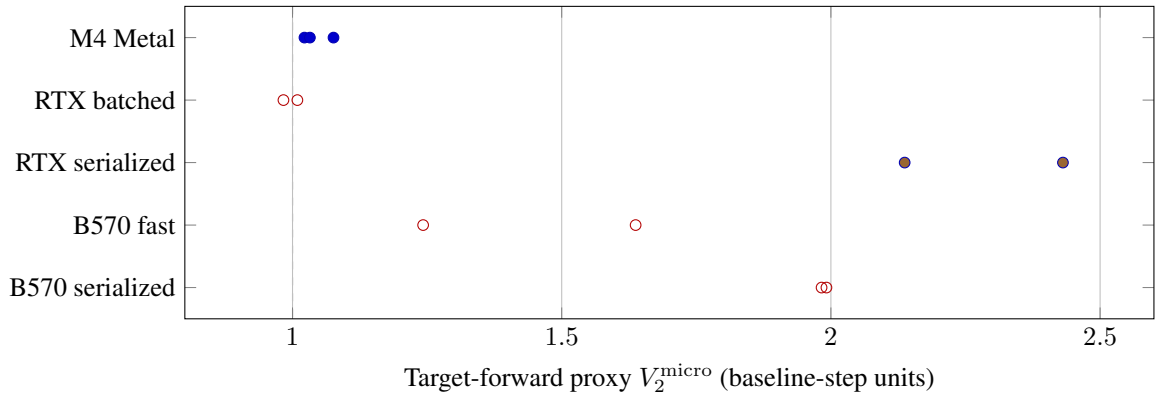


Figure 2. How does correctness change usable verification? Filled marks: paths exact on the frozen reported workloads; open marks: correctness-blocked paths (including B570 serialized). Each mark is one context cell; nearby cells can overlap. M4 contexts: 512/2048/8192; RTX 5070 Ti Laptop depths: 0/256; B570 contexts: 512/2048. Contexts, runtimes, and controls differ: this is not a hardware ranking. The plotted proxy is not complete R_{target}.

The target maintains persistent state, evaluates proposals, removes invalid candidate state when needed, and decodes correction or bonus output. Its complete measured region accounts for 87.34% of CPU-path speculative time. The NPU path instead spends 58.35% in the synchronous OpenVINO call that replays the full committed prefix and generates a proposal. Replay and proposal are inseparable within this measurement; we do not invent a replay-only value. Residual orchestration occupies only 0.04% and 0.02%, so it cannot explain the main loss.

Figure 3 separates isolated steady-state throughput from this realized request decomposition. The isolated NPU draft can outperform the CPU at the tested contexts, but this advantage does not survive the qualified state interface. The conclusion concerns that implementation, not insufficient NPU compute capability. A state-persistent intervention has not been implemented or measured.

Using arithmetic totals at a common boundary reconstructs Equation 2. CPU has $(D_{\text{real}}, R_{\text{target}}, X) = (0.4304, 2.9767, 0.00124)$ and aggregate speedup $0.7227\times$; NPU has $(4.1251, 2.9435, 0.00130)$ and $0.3477\times$. These agree with the ratios of total baseline/speculative times. They intentionally differ from the primary prompt-level geometric means in Table 2.

The earlier Phase-2 screen combined isolated drafting and a target-forward proxy under favorable residual-cost assumptions. It was an eligibility screen, not a validated end-to-end prediction. Phase 3 shows why those proxies must be replaced with complete target-side work and realized stateful draft cost. No end-to-end Phase-3 energy result is claimed, and source reconstruction retains its `STRONGLY_SUPPORTED` qualification.

6 Hardware-Study Synthesis

Favorable local indicators do not translate reliably into integrated gains across the tested consumer configurations. M4's representative and separately preregistered code workloads occupy opposite profitability regimes. That observation stands independently of the two diagnoses.

The path controls and stateful measurements explain two important failures: correctness qualification can remove the cheapest verification path, and a realized state interface can erase an isolated draft-device advantage. The CPU-draft 358H result also cautions that lowering draft cost alone may leave target-side work dominant. These bounded diagnoses do not establish an exhaustive theory of performance.

Figure 4 summarizes profitable exact, exact but unprofitable, physically cheap but correctness-inadmissible, and state-interface-bound cases. Categories overlap and are not hardware ranks. Local promise must be tested against qualified integrated execution at the intended timing boundary. This hardware conclusion requires neither a successful future prototype nor extrapolation to other workloads, stacks, or device families.

7 Runtime Co-Design from Measured Failure Modes

Internal v0.2: proposed design only; implementation and measured results pending. This section reserves approximately 1.5–2 pages of the ten-page planning budget for a possible future measured contribution. It contains requirements, hypotheses, an interface proposal, and evaluation questions only. The hardware study is complete without it. No speed, energy, correctness, implementation, or ablation result is attributed to the proposed design.

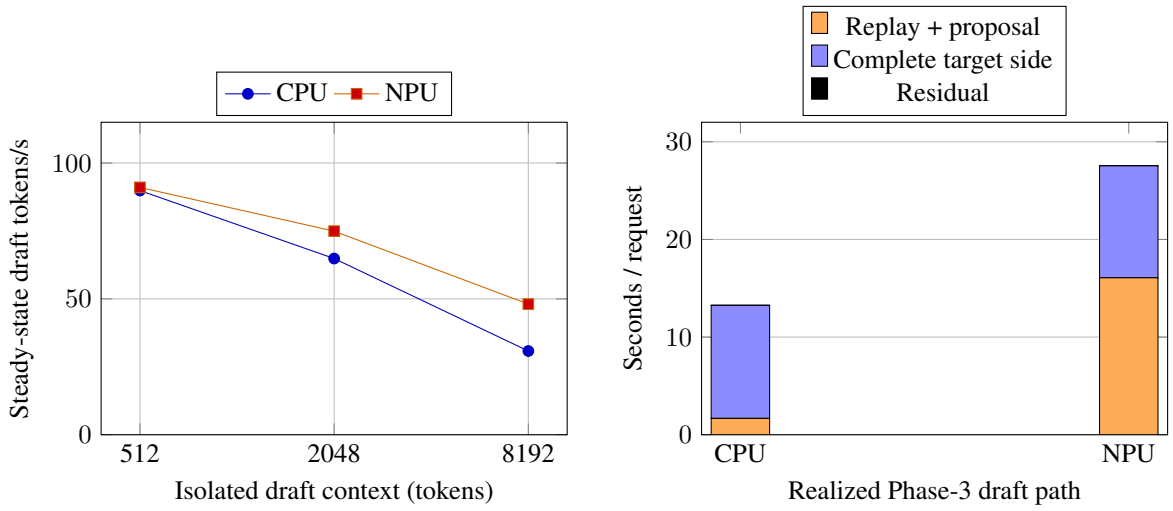


Figure 3. Why does isolated NPU speed not survive integration on 358H? Left: median isolated steady-state draft throughput (five repetitions/cell), excluding the first-token cost. Right: arithmetic mean complete post-prefill request time over 90 observations/path. CPU time is target-side dominated (87.34%); NPU time is replay-plus-proposal dominated (58.35%). Residuals are only 0.04%/0.02% and visually negligible. Separate boundaries preclude interpreting left-panel throughput as realized draft cost.

Profitable + exact
M4 HumanEval: primary

Exact but unprofitable
M4 24Q; 358H CPU/NPU: primary
RTX serialized controls: secondary

Physically cheap, inadmissible
RTX batched; B570 fast: diagnostic

State-interface-bound
358H NPU: measured mechanism

Figure 4. Which observed regimes require explanation? Categories describe outcomes and measured mechanisms; they are neither a device ranking nor an exhaustive causal taxonomy. NPU also belongs to the exact-negative regime. B570 serialized remains correctness-blocked diagnostic evidence, despite its slower execution.

7.1 Requirements traced to closed measurements

The diagnoses suggest targeted experiments rather than a single intervention expected to fix every negative outcome. For the measured 358H interface, a candidate intervention would retain usable draft state across speculative rounds and reconcile it with the committed target history. The NPU replay-plus-proposal region motivates this requirement; the tiny measured residual does not motivate a broad rewrite of the orchestrator. The CPU result additionally warns that reducing draft cost alone may leave complete target-side work dominant. State persistence is therefore a hypothesis about a measured component, not a forecast of overall profitability.

The verification controls impose a separate requirement. A runtime must qualify the target path under its chosen output contract before using that path's timings to select an execution policy. A cheaper blocked path is not a candidate optimization under the current contract. B570 further requires that serialization be recorded as an execution setting, with exactness independently checked across the intended workload. A small probe cannot certify an unrestricted path selection policy.

The M4 contrast motivates an optional low-overhead profitability guard based on realized quantities, without claiming that it diagnoses the cause of that contrast. Its purpose would be to decide whether a qualified speculative path is useful for the current workload, using direct committed progress and complete costs. The guard's own cost would need to be measured. It must not infer realized draft time from an isolated throughput number or count accepted draft tokens as all output progress. The proposed minimum study holds proposal width and device placement fixed; adaptive width and heterogeneous device selection would add separate questions only if later evidence justifies them.

7.2 Minimal state interface proposal

We propose a request-scoped draft-state object associated with a committed token prefix and a model/runtime configuration. A begin operation would initialize state from the prompt. A checkpoint operation would identify the com-

mitted boundary before proposing. A propose operation would return candidate token IDs and a state handle describing their relation to that boundary. A reconcile operation would retain only the accepted draft prefix, incorporate any target correction or bonus token, and discard rejected candidate state. These are proposed interface responsibilities, not available functions in the measured adapter.

The target reference and its correctness contract would stay explicit. After every round, the target logical history, draft logical history, and committed output ledger would have to agree. Candidate state cannot become committed state merely because a proposal call completed. At request termination, all handles must be invalidated so a later request cannot inherit another prefix. If the backend cannot expose safe truncation or restoration, a replay path may be necessary; any such fallback cost must stay inside the realized draft boundary.

The interface should expose a complete draft-call timer, complete target-round timer, residual interval, and committed-token count under one enclosing decode window. Checkpoint creation, state copying, cache allocation, synchronization, and recovery must be charged to the responsible measured region. Merely moving work outside a narrow proposal timer would not establish an improvement. If replay and proposal remain inseparable, the evaluation should keep the aggregate region used by the hardware study rather than claiming an unavailable subcomponent decomposition.

The proposal does not require a new speculative algorithm, a new kernel, or a new learned draft. Its central test is whether the qualified backend can preserve useful state without changing token semantics or paying comparable reconciliation costs. Memory capacity and state-handle lifetime are therefore evaluation questions alongside latency. Persistent state and KV reuse have substantial prior art; their existence is not the intended novelty claim.

7.3 Correctness-qualified path selection

A minimal path registry would associate each target execution setting with its qualification scope: model artifacts, runtime revision, numerical options, context/output conditions, and the evaluated workload. Selection would consider only paths admitted by the active contract. Changing the runtime, quantization, or execution shape would invalidate the relevant qualification rather than inherit it silently. This registry is proposed metadata and policy, not a claim that a finite test suite proves correctness for every possible input.

The evaluation must retain mismatches and blocked candidates. A path that fails remains a diagnostic observation with its failure scope visible. Recovery to target-only execution must preserve the same committed history and have its cost included. A future study could consider a different approximate-quality contract, but it would need its own quality metric and protocol; it cannot retrospectively relabel the frozen failures in this manuscript.

7.4 Planned evaluation questions and decision rule

A separate protocol should be frozen before observing intervention results. It should compare the current replay adapter, the proposed persistent-state adapter, and matched target-only decoding on the same prospective workload and timing boundary. Correctness qualification should exercise accepted prefixes, rejection, correction, bonus output, request reset, and output-budget boundaries. Full-workload exactness must accompany any promoted speed result; passing an initial gate alone is insufficient.

Two additional questions concern the implementation boundary. Does retaining state preserve its benefit as the committed prefix grows, or does checkpoint memory and reconciliation work simply replace replay? Does the same interface handle complete acceptance and early rejection without hidden full-prefix rebuilds? The proposed evaluation would record these events and their charged time. Results could then distinguish a reduction in executed work from a change in where the timer starts. Any context or event breakdown would be descriptive unless specified prospectively as a confirmatory comparison.

A prospective paired comparison should fix the reference, output contract, prefill exclusion, and stopping behavior, randomize run order, and prespecify the primary estimator. Failures, fallback events, and memory pressure must remain visible; selected-subset gains cannot establish workload-wide improvement. These requirements do not assert that a proposed backend API can meet them.

The key questions are whether the interface preserves token equivalence; whether realized draft cost decreases after all state-management work is counted; whether complete target-side cost changes; and whether the end-to-end prompt-level comparison improves. Component measurements and total-time accounting should reconcile. An energy claim would require a newly qualified end-to-end sensor boundary, which the closed 358H campaign does not supply.

Reduced draft cost without overall profitability must be reported as such; correctness failure leaves timing diagnostic. If implementation is unavailable or yields no defensible contribution, this section can be removed without changing the hardware conclusion. A successful intervention would require measured evidence and revised paper claims; none is assumed here.

8 Related Work

Performance and cost accounting. Speculative decoding amortizes target computation while preserving a target sampling distribution under its algorithmic assumptions (Leviathan et al., 2023). Liu et al. (2026) systematically study speculative methods in vLLM, including verification dominance and workload/batch effects. Kim et al. (2026) control shared runtime policies and express speedup through committed work and conditional execution cost. Lyu et al. (2026) derive profitability conditions for edge–cloud execution. Our empirical question concerns how reliably favorable local indicators translate into integrated gains across the tested consumer workloads and stacks. We neither introduce performance characterization nor claim a novel cost/commit identity; the accounting lens supports diagnosis of the observed outcomes.

Heterogeneous execution. Dovetail places drafting on a GPU and target verification on a CPU (Zhang et al., 2025); DuoDecoding studies CPU/GPU placement with hardware-aware draft budgets (Lv et al., 2025). Mirror-SD explicitly considers GPU/NPU mapping and complementary speculative pipelines (Bhendawade et al., 2025). Heterogeneous speculative execution is therefore established prior art. Our negative heterogeneous outcome characterizes the qualified interface and does not contradict acceleration under other placements or runtimes.

Correctness and numerical reproducibility. Zhang et al. (2026) analyze batch speculative-decoding synchronization errors and distinguish them from residual numerical mismatches. Their current paper revises the earlier *Batch Speculative Decoding Done Right* preprint. Yuan et al. (2025) study numerical sources of output variation across system configurations; LLM-42 uses verified speculation to enforce deterministic inference (Gond et al., 2026). Our CUDA observation is not a new numerical phenomenon, and we do not equate it with ragged-batch synchronization bugs. We measure its consequence for an admissible target path, with B570 showing why serialization alone cannot certify the intended workload.

State and memory reuse. Draft-side state management also has prior art. Yuan et al. (2026) maintain incrementally updated compressed draft KV memory for long contexts. Such designs differ from the full-prefix replay interface measured here, but preclude treating state reuse itself as a new idea. Our evidence identifies a realized replay-plus-proposal cost in a consumer OpenVINO NPU path. Any later co-design contribution must be established by its own measured behavior rather than by adopting persistence as an architectural label.

9 Limitations

This study covers four consumer systems and a narrow Qwen3 model family. Hardware, operating systems, runtime versions, backends, and some quantization choices co-vary. The observed failures of local indicators are case evidence, not an estimated population-wide failure rate or a controlled portability benchmark. The diagnosed mechanisms are not exhaustive and do not causally explain the M4 workload contrast. The results establish observed regimes and within-platform mechanisms, not an architectural ranking or a general deployment law. Serving concurrency, production load, and broader model families are outside the measured scope.

Token equivalence is deliberately strict and conditional on the frozen reference; a different quality contract could admit other paths. HumanEval functional execution and pass@k were not evaluated. Serialized B570 outputs still mismatch, and the CUDA evidence is bounded to the captured stack and case. No claim of general backend unsuitability follows.

Campaign-specific estimators and bootstrap implementations differ. Prompt intervals do not cover all system-state uncertainty, including M4 session/thermal effects. The M4 corrected synthetic-ceiling preregistration remains unexecuted. Decode timing excludes initial prefill and loading, so it is not whole-request user latency. Energy sensor domains differ; there is no global energy ranking or end-to-end 358H Phase-3 energy claim.

Windows raw per-run provenance is weaker than that of the other closed campaigns. The 358H draft-source reconstruction is `STRONGLY_SUPPORTED`, not proven historical upstream identity. Replay and proposal are measured together, preventing separate causal timing attribution to either. No measured runtime intervention exists in this version; persistent state, path selection, and profitability control remain proposals. These limitations bound the hardware conclusion independently of the future software decision.

10 Conclusion

Favorable local indicators do not reliably translate into integrated speculative-decoding gains across the tested consumer configurations. M4 exposes both exact positive and exact negative workload-conditioned regimes. Two measured failure modes clarify other outcomes: correctness changes the usable verification path, and realized state-interface cost erases an isolated draft advantage. These diagnoses are not exhaustive. The hardware conclusion is bounded to the frozen workloads, stacks, and post-prefill decode measurements, and stands without a new runtime result. The final paper conclusion remains provisional pending separately measured software evidence or removal of that reserved scope.

REFERENCES

Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https://aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.

Zhang, R. H., Dey, S., Mishra, A., Wu, H., Li, B., and Zhang, R. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. *arXiv preprint arXiv:2510.22876v4*, 2026. URL https://arxiv.org/abs/2510.22876v4. Accepted to Findings of EMNLP 2026.